Dynamic Programming - Optimization Problems

0/1 Knapsack, Travelling Salesperson Problem (DP version), Optimal Binary Search Tree

Exercise 1: 0/1 Knapsack Problem (Dynamic Programming)

Aim:

To write a Python program that finds the maximum profit obtainable by selecting items into a knapsack of limited capacity, where each item can be taken at most once, using dynamic programming (tabulation).

Algorithm:

1. Create a 2-D table dp[i][w] where i ranges over the items (0 to n) and w ranges over the capacities (0 to W), initialized to 0.
2. For each item i from 1 to n and each capacity w from 0 to W: if the weight of item i is less than or equal to w, set dp[i][w] = max(value[i-1] + dp[i-1][w-weight[i-1]], dp[i-1][w]); this means the item can either be included or excluded, and the better of the two is chosen.
3. If the weight of item i exceeds w, the item cannot be included, so set dp[i][w] = dp[i-1][w].
4. Repeat step 2-3 for all items and all capacities to fill the table.
5. Return dp[n][W], which holds the maximum profit obtainable within capacity W using the first n items, each of which can be taken at most once.

Python Code:

```
def knapsack(weights, values, capacity):
    n = len(weights)
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(
                    values[i - 1] + dp[i - 1][w - weights[i - 1]],
                    dp[i - 1][w]
                )
            else:
                dp[i][w] = dp[i - 1][w]
    return dp[n][capacity]

weights = [1, 3, 4, 5]
values = [1, 4, 5, 7]
capacity = 7
print("Maximum Profit:", knapsack(weights, values, capacity))
```

Input:

```
weights = [1, 3, 4, 5]
values = [1, 4, 5, 7]
capacity = 7
```

Output:

```
Maximum Profit: 9
```

Result:

The Python program was executed successfully and the maximum profit obtainable within a knapsack capacity of 7 was found to be 9, using dynamic programming tabulation.

Exercise 2: Travelling Salesperson Problem (Dynamic Programming)

Aim:

To write a Python program that finds the minimum cost of visiting all cities exactly once and returning to the starting city using dynamic programming with bitmasking (Held-Karp algorithm).

Algorithm:

1. Represent the state as (mask, pos), where mask is a bitmask denoting the set of cities already visited and pos is the current city.
2. If mask includes all cities (mask == (1<<n) - 1), return the cost of returning from the current city to the starting city.
3. Otherwise, for every unvisited city, recursively compute the cost of visiting it next and add the edge cost from the current city to that city.
4. Take the minimum cost among all these choices and store/reuse it using memoization (via mask and pos) to avoid recomputation of the same sub-problem.
5. Start the recursion from mask = 1 (only the starting city visited) and pos = 0, and return the resulting minimum tour cost.

Python Code:

```
from functools import lru_cache

def tsp(graph):
    n = len(graph)
    @lru_cache(None)
    def visit(mask, pos):
        if mask == (1 << n) - 1:
            return graph[pos][0]
        ans = float('inf')
        for city in range(n):
            if mask & (1 << city) == 0:
                ans = min(
                    ans, graph[pos][city] + visit(mask | (1 << city), city)
                )
        return ans
    return visit(1, 0)

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
print("Minimum Cost:", tsp(graph))
```

Input:

```
graph = [
  [0, 10, 15, 20],
  [10, 0, 35, 25],
  [15, 35, 0, 30],
```

```
  [20, 25, 30, 0]
]
```

Output:

Minimum Cost: 80

Result:

The Python program was executed successfully and the minimum cost tour that visits every city exactly once and returns to the start was found to be 80.

Exercise 3: Optimal Binary Search Tree (OBST)

Aim:

To write a Python program that constructs a binary search tree from a set of keys and their search frequencies such that the total search cost is minimized, using dynamic programming.

Algorithm:

1. Create a 2-D cost table cost[i][j] and initialize cost[i][i] = freq[i] for every single key, since a tree with one key costs just its own frequency.
2. For every chain length L from 2 to n, and every starting index i (with j = i + L - 1 as the ending index), consider each key r between i and j as the root of the subtree covering keys i to j.
3. For each choice of root r, compute the cost as cost[i][r-1] (left subtree, or 0 if r = i) + cost[r+1][j] (right subtree, or 0 if r = j) + the sum of frequencies from i to j (since every key in the subtree is one level deeper when its root is elsewhere).
4. Set cost[i][j] to the minimum cost obtained over all choices of root r.
5. After processing all chain lengths, return cost[0][n-1], which is the minimum cost of the optimal binary search tree over all n keys.

Python Code:

```
def optimal_bst(keys, freq):
    n = len(keys)
    cost = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        cost[i][i] = freq[i]
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            cost[i][j] = float('inf')
            total_freq = sum(freq[i:j + 1])
            for r in range(i, j + 1):
                left = cost[i][r - 1] if r > i else 0
                right = cost[r + 1][j] if r < j else 0
                cost[i][j] = min(
                    cost[i][j],
                    left + right + total_freq
                )
    return cost[0][n - 1]

keys = [10, 20, 30]
freq = [34, 8, 50]
print("Optimal BST Cost:", optimal_bst(keys, freq))
```

Input:

```
keys = [10, 20, 30]
freq = [34, 8, 50]
```

Output:

```
Optimal BST Cost: 142
```

Result:

The Python program was executed successfully and the minimum search cost of the optimal binary search tree built on keys 10, 20 and 30 was found to be 142.

Exercise 4: 0/1 Knapsack Problem

Aim:

To write a Python program that finds the maximum profit obtainable by selecting items into a knapsack of limited weight capacity, where each item can be taken at most once, using the bottom-up dynamic programming approach.

Algorithm:

1. Create a 2-D table dp[i][w] where i ranges over the items (0 to n) and w ranges over the capacities (0 to W), initialized to 0.

2. For each item i from 1 to n and each capacity w from 0 to W: if the weight of item i is less than or equal to w, set dp[i][w] = max(value[i-1] + dp[i-1][w-weight[i-1]], dp[i-1][w]); this means the item can either be included or excluded, and the better of the two is chosen.

3. If the weight of item i exceeds w, the item cannot be included, so set dp[i][w] = dp[i-1][w].

4. Repeat step 2-3 for all items and all capacities to fill the table.

5. Return dp[n][W], which holds the maximum profit obtainable within capacity W using the first n items, each of which can be taken at most once.

Python Code:

```
def knapsack(wt, val, W):
    n = len(wt)
    dp = [[0]*(W+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(W+1):
            if wt[i-1] <= w:
                dp[i][w] = max(val[i-1] + dp[i-1][w-wt[i-1]],
                               dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]
    return dp[n][W]

wt = [1, 3, 4, 5]
val = [1, 4, 5, 7]
W = 7
print("Maximum Profit =", knapsack(wt, val, W))
```

Input:

```
wt = [1, 3, 4, 5]
val = [1, 4, 5, 7]
W = 7
```

Output:

```
Maximum Profit = 9
```

Result:

The Python program was executed successfully and the maximum profit obtainable within a knapsack weight capacity of 7 was confirmed to be 9.

Exercise 5: 0/1 Knapsack Using Memoization

Aim:

To write a Python program that finds the maximum profit obtainable in the 0/1 knapsack problem using the top-down dynamic programming approach with memoization.

Algorithm:

1. Define a recursive function knapsack(W, wt, val, n, memo) that returns the maximum value obtainable using the first n items and capacity W.
2. If n = 0 or W = 0, return 0, since no items or no capacity means no value can be obtained.
3. If the result for the current (n, W) state has already been computed and stored in memo, return the stored value directly instead of recomputing it.
4. If the weight of the n-th item is less than or equal to W, compute the maximum of including the item (its value plus the result for the remaining capacity and items) and excluding it (the result using one fewer item); store this in memo[n][W].
5. If the weight of the n-th item exceeds W, the item cannot be included, so store the result of excluding it in memo[n][W].
6. Return memo[n][W] as the maximum achievable profit.

Python Code:

```
def knapsack(W, wt, val, n, memo):
    if n == 0 or W == 0:
        return 0
    if memo[n][W] != -1:
        return memo[n][W]
    if wt[n-1] <= W:
        memo[n][W] = max(
            val[n-1] + knapsack(W-wt[n-1], wt, val, n-1, memo),
            knapsack(W, wt, val, n-1, memo)
        )
    else:
        memo[n][W] = knapsack(W, wt, val, n-1, memo)
    return memo[n][W]

wt = [1, 3, 4, 5]
val = [1, 4, 5, 7]
W = 7
memo = [[-1]*(W+1) for _ in range(len(wt)+1)]
print("Maximum Profit =", knapsack(W, wt, val, len(wt), memo))
```

Input:

```
wt = [1, 3, 4, 5]
val = [1, 4, 5, 7]
W = 7
```

Output:

```
Maximum Profit = 9
```

Result:

The Python program was executed successfully and the memoized top-down approach also correctly produced a maximum profit of 9, confirming the tabulation-based result.

Exercise 6: Subset Sum Problem

Aim:

To write a Python program that determines whether there exists a subset of a given array whose elements sum exactly to a given target value, using dynamic programming.

Algorithm:

1. Create a 2-D boolean table dp[i][j] where i ranges over the array elements (0 to n) and j ranges over possible sums (0 to target).

2. Set dp[i][0] = True for all i, since a sum of 0 is always achievable using an empty subset.

3. For each element i from 1 to n and each sum j from 1 to target: if the element arr[i-1] is less than or equal to j, set dp[i][j] = dp[i-1][j] OR dp[i-1][j - arr[i-1]] (either exclude or include the element).

4. If arr[i-1] is greater than j, set dp[i][j] = dp[i-1][j], since the element cannot be included.

5. Return dp[n][target], which is True if some subset of the array sums exactly to the target, and False otherwise.

Python Code:

```
def subset_sum(arr, target):
    n = len(arr)
    dp = [[False]*(target+1) for _ in range(n+1)]
    for i in range(n+1):
        dp[i][0] = True
    for i in range(1, n+1):
        for j in range(1, target+1):
            if arr[i-1] <= j:
                dp[i][j] = dp[i-1][j] or dp[i-1][j-arr[i-1]]
            else:
                dp[i][j] = dp[i-1][j]
    return dp[n][target]

print(subset_sum([3, 34, 4, 12, 5, 2], 9))
```

Input:

arr = [3, 34, 4, 12, 5, 2]
target = 9

Output:

True

Result:

The Python program was executed successfully and correctly determined that a subset (4 and 5) of the given array sums exactly to the target value of 9.

Exercise 7: Travelling Salesperson Problem (DP Version)

Aim:

To write a Python program that solves the Travelling Salesperson Problem using dynamic programming with bitmasking and memoization to find the minimum cost tour.

Algorithm:

1. Represent the state as (mask, pos), where mask is a bitmask denoting the set of cities already visited and pos is the current city.
2. If mask includes all cities (mask == (1<<n) - 1), return the cost of returning from the current city to the starting city.
3. Otherwise, for every unvisited city, recursively compute the cost of visiting it next and add the edge cost from the current city to that city.
4. Take the minimum cost among all these choices and store/reuse it using memoization (via mask and pos) to avoid recomputation of the same sub-problem.
5. Start the recursion from mask = 1 (only the starting city visited) and pos = 0, and return the resulting minimum tour cost.

Python Code:

```
from functools import lru_cache

graph = [
   [0,10,15,20],
   [10,0,35,25],
   [15,35,0,30],
   [20,25,30,0]
]
n = len(graph)

@lru_cache(None)
def tsp(mask, pos):
   if mask == (1<<n)-1:
      return graph[pos][0]
   ans = float('inf')
   for city in range(n):
      if not mask & (1<<city):
         ans = min(ans,
                   graph[pos][city] +
                   tsp(mask | (1<<city), city))
   return ans

print("Minimum Cost =", tsp(1, 0))
```

Input:

```
graph = [
  [0,10,15,20],
  [10,0,35,25],
  [15,35,0,30],
  [20,25,30,0]
]
```

Output:

Minimum Cost = 80

Result:

The Python program was executed successfully and the minimum cost tour was again found to be 80, confirming the result obtained by the earlier recursive formulation.

Exercise 8: Travelling Salesperson Using DP Table

Aim:

To write a Python program that solves the Travelling Salesperson Problem using an iterative bottom-up dynamic programming table indexed by bitmask and current city.

Algorithm:

1. Create a 2-D table dp[mask][u], where mask represents the set of visited cities and u is the current city, initialized to infinity, except dp[1][0] = 0 (start at city 0 with only city 0 visited).
2. For every mask from 0 to 2^n - 1 and every city u included in mask, try extending the path to every city v not yet in mask.
3. Update dp[mask | (1<<v)][v] = min(dp[mask | (1<<v)][v], dp[mask][u] + graph[u][v]), representing the cost of visiting v next.
4. After processing all masks, for every city i (other than the start), compute dp[(1<<n)-1][i] + graph[i][0], the cost of completing the tour by returning to the start.
5. Return the minimum value among these completions as the minimum cost of the tour.

Python Code:

```
def tsp_dp(graph):
    n = len(graph)
    dp = [[float('inf')]*n for _ in range(1<<n)]
    dp[1][0] = 0
    for mask in range(1<<n):
        for u in range(n):
            if mask & (1<<u):
                for v in range(n):
                    if not mask & (1<<v):
                        dp[mask | (1<<v)][v] = min(
                            dp[mask | (1<<v)][v],
                            dp[mask][u] + graph[u][v]
                        )
    ans = float('inf')
    for i in range(1, n):
        ans = min(ans, dp[(1<<n)-1][i] + graph[i][0])
    return ans

graph = [
    [0,10,15,20],
    [10,0,35,25],
    [15,35,0,30],
    [20,25,30,0]
]
print("Minimum Cost =", tsp_dp(graph))
```

Input:

```
graph = [
  [0,10,15,20],
  [10,0,35,25],
  [15,35,0,30],
  [20,25,30,0]
]
```

Output:

Minimum Cost = 80

Result:

The Python program was executed successfully and the iterative DP table approach also correctly produced a minimum tour cost of 80, matching the memoized recursive solutions.

Exercise 9: Optimal Binary Search Tree (OBST)

Aim:

To write a Python program that computes the minimum search cost of an optimal binary search tree given only the key frequencies, using dynamic programming.

Algorithm:

1. Create a 2-D cost table cost[i][j] and initialize cost[i][i] = freq[i] for every single key, since a tree with one key costs just its own frequency.

2. For every chain length L from 2 to n, and every starting index i (with j = i + L - 1 as the ending index), consider each key r between i and j as the root of the subtree covering keys i to j.

3. For each choice of root r, compute the cost as cost[i][r-1] (left subtree, or 0 if r = i) + cost[r+1][j] (right subtree, or 0 if r = j) + the sum of frequencies from i to j (since every key in the subtree is one level deeper when its root is elsewhere).

4. Set cost[i][j] to the minimum cost obtained over all choices of root r.

5. After processing all chain lengths, return cost[0][n-1], which is the minimum cost of the optimal binary search tree over all n keys.

Python Code:

```
def optimal_bst(freq):
    n = len(freq)
    cost = [[0]*n for _ in range(n)]
    for i in range(n):
        cost[i][i] = freq[i]
    for L in range(2, n+1):
        for i in range(n-L+1):
            j = i+L-1
            cost[i][j] = float('inf')
            total = sum(freq[i:j+1])
            for r in range(i, j+1):
                left = cost[i][r-1] if r > i else 0
                right = cost[r+1][j] if r < j else 0
                cost[i][j] = min(cost[i][j],
                                 left + right + total)
    return cost[0][n-1]

print("Optimal Cost =", optimal_bst([34, 8, 50]))
```

Input:

freq = [34, 8, 50]

Output:

Optimal Cost = 142

Result:

The Python program was executed successfully and the minimum search cost of the optimal binary search tree was again confirmed to be 142.

Exercise 10: Matrix Chain Multiplication

Aim:

To write a Python program that finds the minimum number of scalar multiplications needed to multiply a chain of matrices in the optimal order, using dynamic programming.

Algorithm:

1. Create a 2-D table dp[i][j] representing the minimum number of scalar multiplications needed to compute the matrix chain product from matrix i to matrix j.
2. For every chain length L from 2 to n-1, and every starting index i (with j = i + L - 1), initialize dp[i][j] to infinity.
3. For every possible split point k between i and j, compute q = dp[i][k] + dp[k+1][j] + arr[i-1] * arr[k] * arr[j], which is the cost of splitting the chain at k.
4. Set dp[i][j] to the minimum value of q over all split points k.
5. Repeat for all chain lengths and return dp[1][n-1], the minimum number of multiplications needed to multiply the entire chain of matrices.

Python Code:

```
def matrix_chain(arr):
    n = len(arr)
    dp = [[0]*n for _ in range(n)]
    for L in range(2, n):
        for i in range(1, n-L+1):
            j = i+L-1
            dp[i][j] = float('inf')
            for k in range(i, j):
                q = dp[i][k] + dp[k+1][j] + arr[i-1]*arr[k]*arr[j]
                dp[i][j] = min(dp[i][j], q)
    return dp[1][n-1]

print("Minimum Multiplications =", matrix_chain([10, 20, 30, 40, 30]))
```

Input:

arr (matrix dimensions) = [10, 20, 30, 40, 30]

Output:

Minimum Multiplications = 30000

Result:

The Python program was executed successfully and the minimum number of scalar multiplications required to multiply the chain of matrices in the optimal order was found to be 30000.

Exercise 11: Coin Change (Minimum Coins)

Aim:

To write a Python program that finds the minimum number of coins required to make up a given amount using dynamic programming.

Algorithm:

1. Create a 1-D table dp[] of size (amount + 1), initialize dp[0] = 0 and all other entries to infinity.
2. For every amount i from 1 to the target amount, and for every coin denomination c that is less than or equal to i, update dp[i] = min(dp[i], dp[i-c] + 1).
3. This means the minimum coins needed for amount i is the minimum, over all coins c, of one plus the minimum coins needed for the remaining amount (i - c).
4. Repeat this process for all amounts up to the target amount.
5. Return dp[amount], which holds the minimum number of coins required to make up the given amount.

Python Code:

```
def min_coins(coins, amount):
    dp = [float('inf')] * (amount+1)
    dp[0] = 0
    for i in range(1, amount+1):
        for c in coins:
            if c <= i:
                dp[i] = min(dp[i], dp[i-c]+1)
    return dp[amount]

print("Minimum Coins =", min_coins([1, 2, 5], 11))
```

Input:

```
coins = [1, 2, 5]
amount = 11
```

Output:

```
Minimum Coins = 3
```

Result:

The Python program was executed successfully and the minimum number of coins needed to make up an amount of 11 using the dynamic programming approach was found to be 3.

Exercise 12: Rod Cutting Problem

Aim:

To write a Python program that finds the maximum revenue obtainable by cutting a rod of given length into pieces and selling them, using dynamic programming.

Algorithm:

1. Create a 1-D table dp[] of size (n + 1), initialized to 0, where dp[i] represents the maximum revenue obtainable from a rod of length i.
2. For every rod length i from 1 to n, try every possible first cut j from 0 to i-1.
3. For each cut j, compute the revenue as price[j] (revenue from a piece of length j+1) plus dp[i-j-1] (the best revenue from the remaining length).
4. Set dp[i] to the maximum revenue obtained over all possible first cuts j.
5. Return dp[n], which holds the maximum revenue obtainable by cutting a rod of length n optimally.

Python Code:

```
def rod_cut(price, n):
    dp = [0]*(n+1)
    for i in range(1, n+1):
        for j in range(i):
            dp[i] = max(dp[i],
                    price[j] + dp[i-j-1])
    return dp[n]

print("Maximum Revenue =", rod_cut([1, 5, 8, 9, 10, 17, 17, 20], 8))
```

Input:

```
price = [1, 5, 8, 9, 10, 17, 17, 20]
n = 8
```

Output:

```
Maximum Revenue = 22
```

Result:

The Python program was executed successfully and the maximum revenue obtainable by optimally cutting a rod of length 8 was found to be 22.

Exercise 13: Minimum Cost Path

Aim:

To write a Python program that finds the minimum cost path from the top-left cell to the bottom-right cell of a cost grid, where movement is allowed only rightward or downward, using dynamic programming.

Algorithm:

1. Create a 2-D table dp[][] of the same dimensions as the cost matrix, and set dp[0][0] = cost[0][0].
2. Fill the first column: dp[i][0] = dp[i-1][0] + cost[i][0] for all rows, since the first column can only be reached by moving down.
3. Fill the first row: dp[0][j] = dp[0][j-1] + cost[0][j] for all columns, since the first row can only be reached by moving right.
4. For every other cell (i, j), set dp[i][j] = cost[i][j] + min(dp[i-1][j], dp[i][j-1]), choosing the cheaper of arriving from above or from the left.
5. Return dp[m-1][n-1], which is the minimum cost of a path from the top-left cell to the bottom-right cell of the grid.

Python Code:

```
def min_cost(cost):
    m = len(cost)
    n = len(cost[0])
    dp = [[0]*n for _ in range(m)]
    dp[0][0] = cost[0][0]
    for i in range(1, m):
        dp[i][0] = dp[i-1][0] + cost[i][0]
    for j in range(1, n):
        dp[0][j] = dp[0][j-1] + cost[0][j]
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = cost[i][j] + min(dp[i-1][j],
                                        dp[i][j-1])
    return dp[m-1][n-1]

cost = [
    [1, 3, 1],
    [1, 5, 1],
    [4, 2, 1]
]
print("Minimum Cost =", min_cost(cost))
```

Input:

```
cost = [
  [1, 3, 1],
  [1, 5, 1],
  [4, 2, 1]
]
```

Output:

```
Minimum Cost = 7
```

Result:

The Python program was executed successfully and the minimum cost path from the top-left to the bottom-right cell of the grid, moving only right or down, was found to be 7.